

CS 201 — Data Structures and Algorithms

Handout 5: Big-O Notation

Prof. Jane Doe · May 2, 2026

2 Common Complexities

Complexity	Name	$n=1000$ ops
$O(1)$	Constant	1
$O(\log n)$	Logarithmic	≈ 10
$O(n)$	Linear	1 000
$O(n \log n)$	Log-linear	$\approx 10\,000$
$O(n^2)$	Quadratic	1 000 000
$O(2^n)$	Exponential	$\approx 10^{301}$
$O(n!)$	Factorial	Astronomical

1 Introduction

Big-O notation describes the upper bound of an algorithm's time or space complexity as a function of input size n . It answers the question: *how does runtime grow as input grows?*

Formally, $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Formal Definition

$f(n) = O(g(n))$ iff $\exists c > 0, n_0 \geq 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Why Drop Constants and Lower Terms?

Big-O captures *growth rate*, not exact counts. Constants and lower-order terms become negligible as $n \rightarrow \infty$, so e.g. $3n^2 + 5n + 7 = O(n^2)$.

Best, Average, Worst Case

Big-O typically describes the **worst case**. Omega (Ω) gives the lower bound; Theta (Θ) gives a tight bound (both upper and lower).

Amortized Analysis

Some operations are expensive occasionally but cheap on average. E.g. dynamic array append is $O(n)$ worst case but $O(1)$ *amortized* over a sequence of operations.

Remember: When comparing algorithms, prefer lower Big-O classes

3 Quick Reference Rules

- **Loops:** Nested loops multiply — k nested loops $\Rightarrow O(n^k)$.
- **Sequential code:** Add complexities, keep the dominant term.
- **Divide and conquer:** $T(n) = aT(n/b) + O(n^d)$ — use the Master Theorem.
- **Logarithmic factors** arise from halving/doubling (binary search, balanced trees).

4 Practice Problems

Determine the Big-O complexity of each:

1. **Sum of elements:** for i in range(n):
 $s += a[i]$
2. **Nested loops:** for i in range(n):
 for j in range(n):
 print($a[i][j]$)
3. **Binary search** on a sorted array of size n .
4. **Merge sort:** $T(n) = 2T(n/2) + O(n)$.
5. **Triple nested:** for i in range(n):
 for j in range(i):

```
for k in range(j): count += 1
```

5 Further Reading

- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- Sedgewick, Wayne. *Algorithms*, 4th ed., Addison-Wesley, 2011.
- Sipser, M. *Introduction to the Theory of Computation*, 3rd ed., Cengage, 2012.